



Designing Artificial Intelligence Products

Week 2 Office Hour
7 Sep 2026

Danny Malter



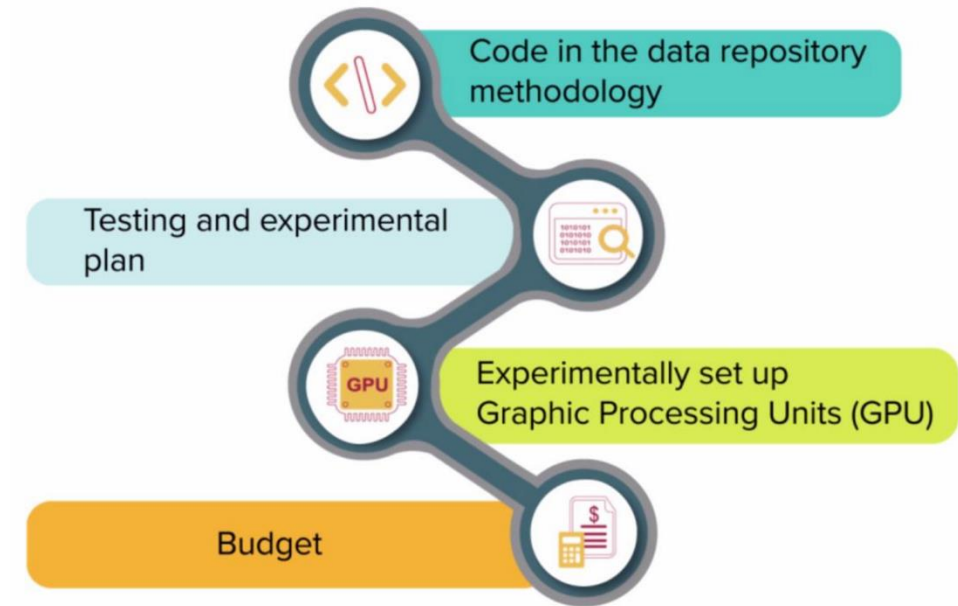
Review Week 2

Questions

What are all the things we need to worry about when we think about the Software development plan?

- **Versioning & Reproducibility**
Code, models, prompts, and datasets change over time
- **Infrastructure & Cost Management**
Unexpected GPU/API costs can grow quickly
- **Pipeline & Workflow Management**
AI systems often involve multiple connected components
Example: SQL → Python → Model → Dashboard
Use orchestration tools like Apache Airflow
- **Production-Level Engineering**
AI prototypes must be converted into scalable, maintainable systems
- **AI-Specific Challenges**
Model drift, hallucinations, bias, and data quality issues
Continuous monitoring and retraining may be required

Key Points in Software Development



Week 2 Questions

1

What does the data acquisition process usually look like?



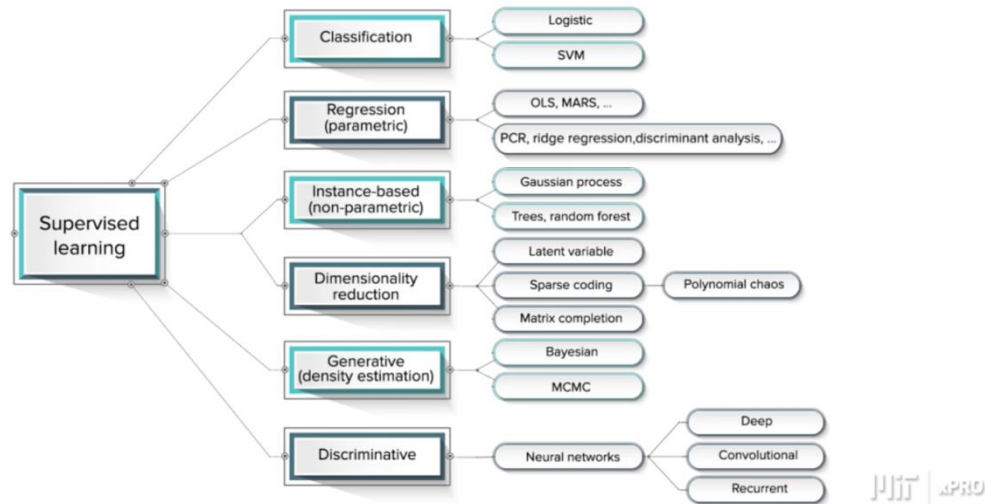
2

Is it possible to acquire pre-annotated data?

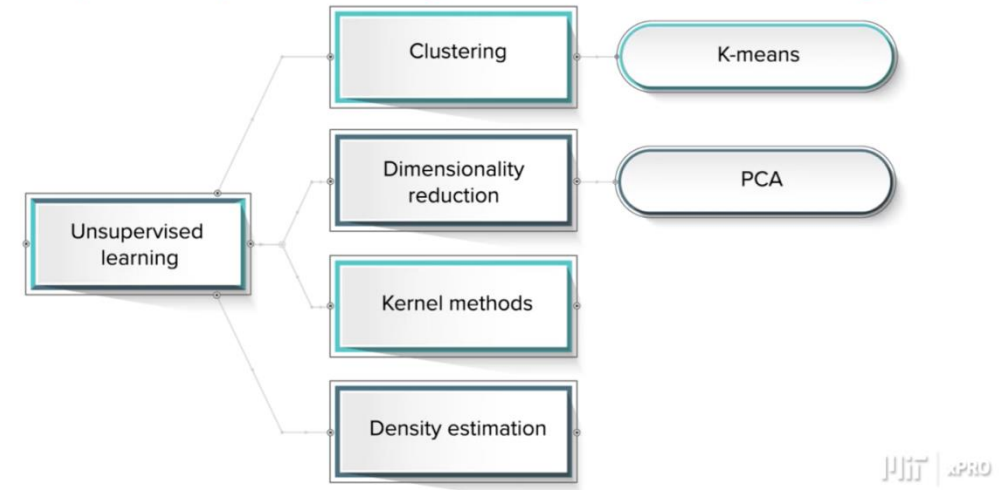


Video 1: Overview and Taxonomy

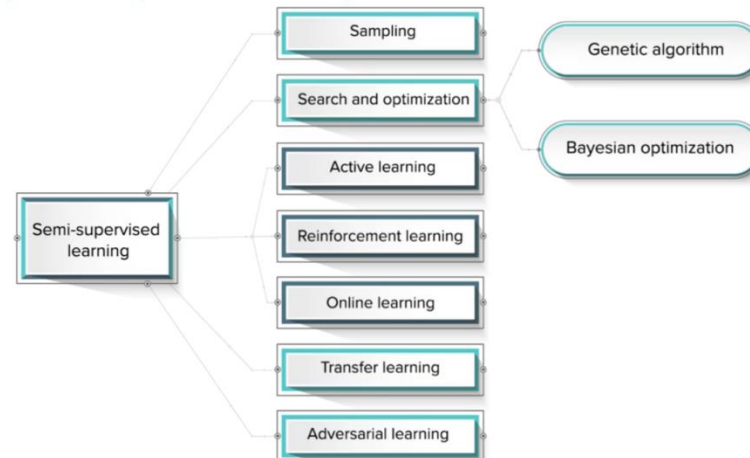
(Partial) Taxonomy of Machine Learning



(Partial) Taxonomy of Machine Learning



(Partial) Taxonomy of Machine Learning



Structured Data

Data organized in rows, columns, and defined fields — the schema is explicit. Easiest for AI to consume directly.

Examples



Relational tables

Customer records, orders, inventory in SQL databases



Churn prediction, credit scoring, customer lifetime value (classical ML)



Spreadsheets / CSVs

Financials, sales reports, marketing performance exports



Forecasting, segmentation, A/B test analysis with tabular models



Transaction logs

Card payments, web events, point-of-sale records



Real-time fraud detection, recommendation engines, anomaly alerts



Sensor / time-series

IoT readings, machine telemetry, stock prices



Predictive maintenance, demand forecasting, algorithmic trading

Unstructured Data

Data with no fixed schema — raw text, images, audio, video. AI turns it into structured signal.

Examples



Text

Reviews, emails, support tickets, contracts



What AI does with it

Sentiment analysis, summarization, chatbot answers (LLMs)



Images

Product photos, X-rays, satellite imagery



Visual search, defect detection, medical screening (CV)



Audio

Call recordings, voice notes, podcasts



Transcription, voice assistants, fraud detection by tone



Video

Surveillance feeds, training footage, social clips



Activity recognition, content moderation, auto-tagging

Supervised vs Unsupervised

What does it mean to have labeled data?

Unlabeled data

Age	Income	Tenure
34	\$72k	2 yr
52	\$110k	8 yr
29	\$58k	1 yr
41	\$95k	5 yr

Features only — no outcome to learn from

Labeled data

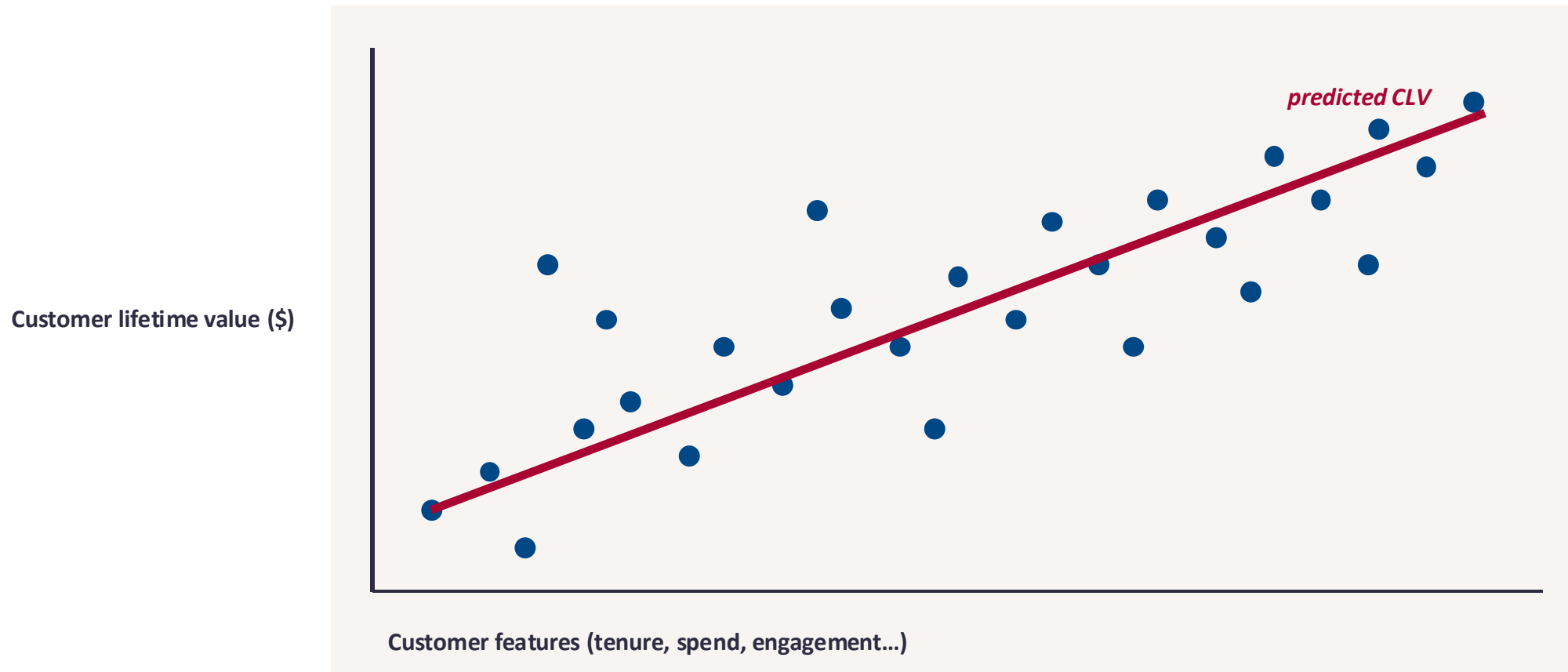
Age	Income	Tenure	Churned?
34	\$72k	2 yr	Yes
52	\$110k	8 yr	No
29	\$58k	1 yr	Yes
41	\$95k	5 yr	No

Each row carries a known target (the label) we can train against

Supervised vs Unsupervised

Marketing wants to predict each customer's lifetime value.

Supervised or unsupervised learning?

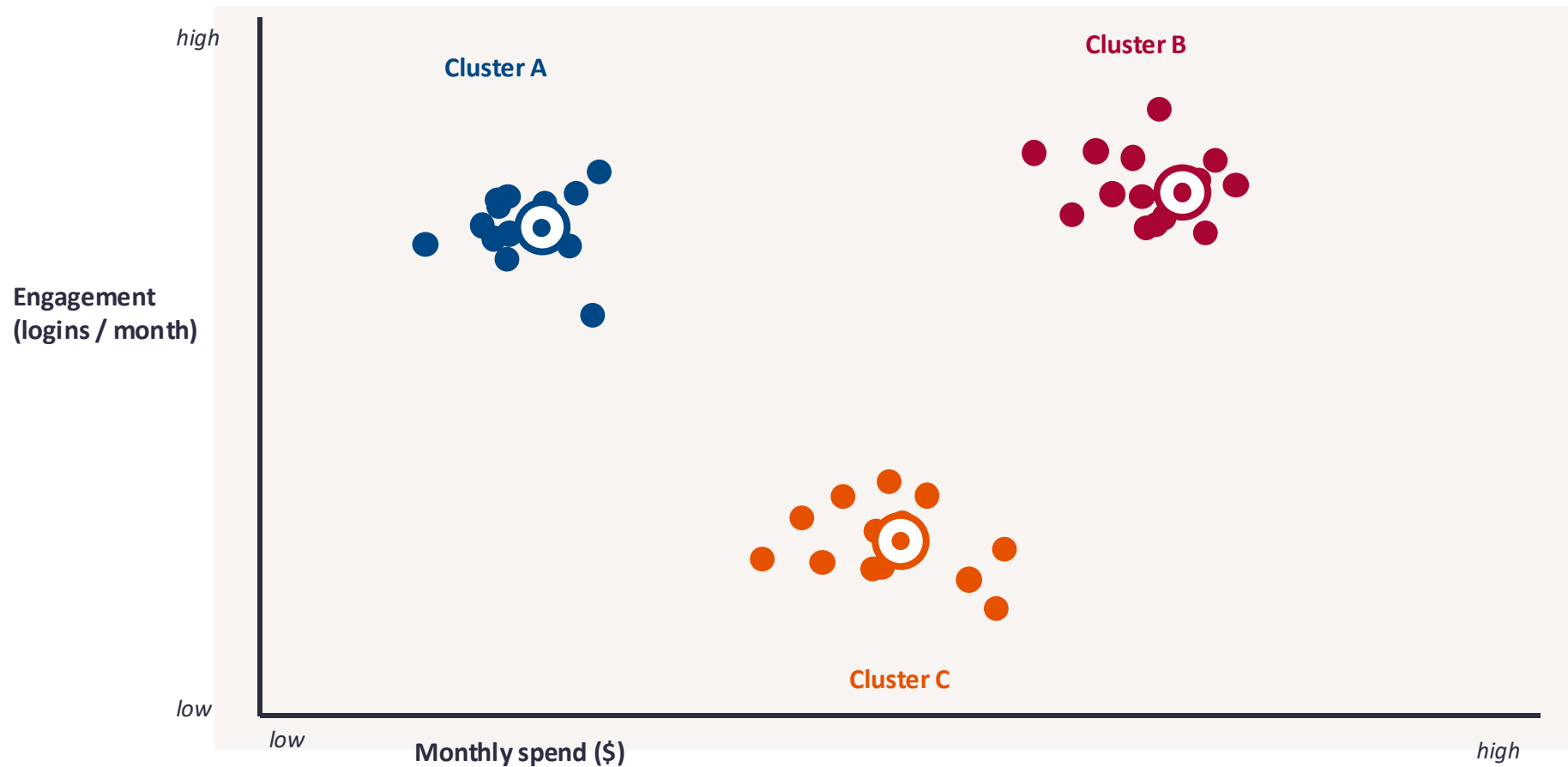


Each customer has a known \$ value → predict a continuous number

Supervised vs Unsupervised

Marketing groups users into segments using k-means clustering.

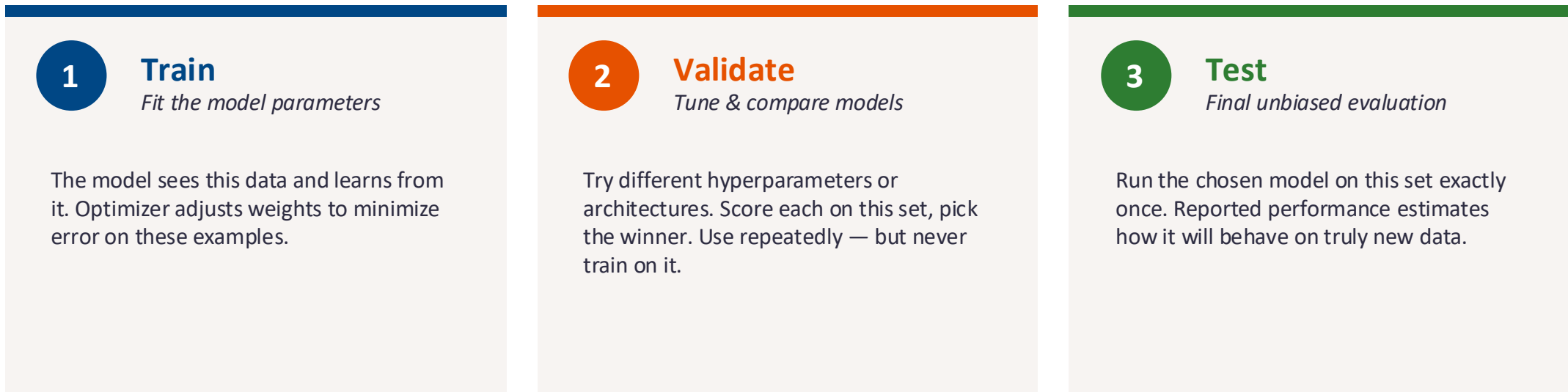
Supervised or unsupervised learning?



Train, Validate, Test

Split the dataset once — each split serves a different purpose, and the test set is held back until the very end.

Available data (100%)



Common splits: 60/20/20 (shown) · 70/15/15 · 80/10/10. With small datasets, use k -fold cross-validation instead of a fixed validation set.

Scoring the Model on Test Data

After training, the model is just a formula. Apply it to test rows, compare predictions to actuals, and score the gap.

The trained model (housing prices, \$k)

$$\hat{y} = 0.06 \cdot \text{sqft} + 0.30 \cdot \text{bedrooms} - 1.2 \cdot \text{age} + 50$$

Coefficients learned from the 60% training rows — they don't change going forward.

How we score

error = actual - predicted

MAE = mean of |errors|

RMSE = $\sqrt{\text{mean of errors}^2}$

Apply formula to held-out test rows:

sqft	bed	age	Actual (\$k)	Predicted (\$k)	Error
1,500	3	10	130.0	128.9	-1.1
2,200	4	5	180.0	177.2	-2.8
1,800	3	20	140.0	134.9	-5.1
2,500	5	2	210.0	199.1	-10.9
1,200	2	30	95.0	86.6	-8.4

MAE

\$5.66k

Avg dollar error per home

RMSE

\$6.70k

Penalizes big misses more

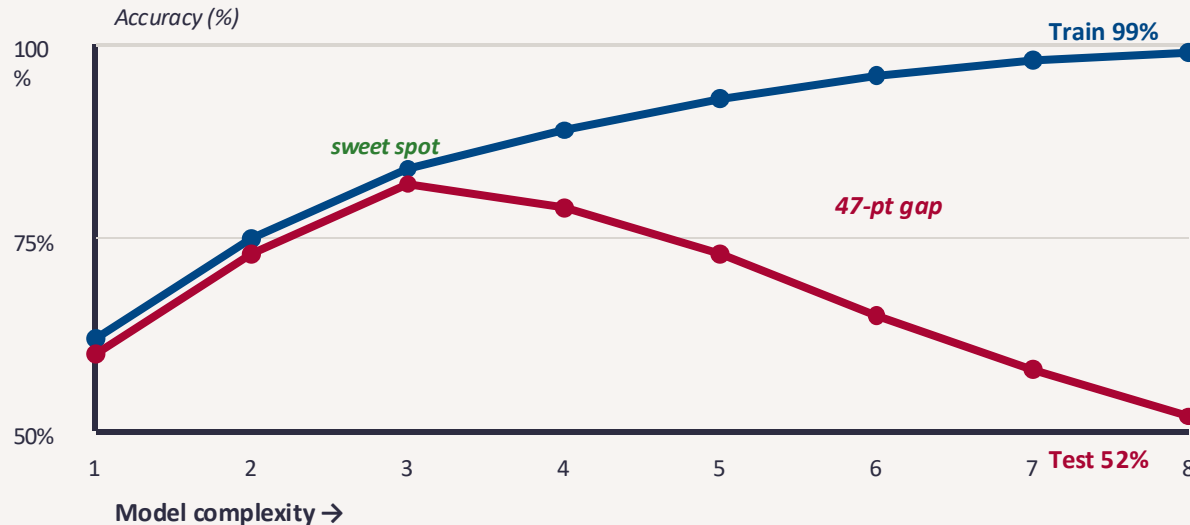
So what?

On unseen homes, this model's price predictions are off by ~\$6k on average. Whether that's good enough depends on the business stakes.

Overfitting — why we hold out test data

A model that memorizes the training data won't generalize. The test set is the only honest way to catch this.

Accuracy vs model complexity



Three points on the curve

Complexity	Train Acc	Test Acc	Verdict
1	62%	60%	Underfit
3	84%	82%	Good fit
8	99%	52%	Overfit

As complexity grows, training accuracy keeps climbing — but test accuracy peaks and then falls.

Why the split matters

Training error always looks good — the model has seen every answer. Held-out data is the only way to know if the model actually learned the pattern or just memorized the rows.

Signs of overfitting

Training error keeps dropping while test error rises. Performance falls off a cliff on new data. The model picks up noise as if it were signal.

How to combat it

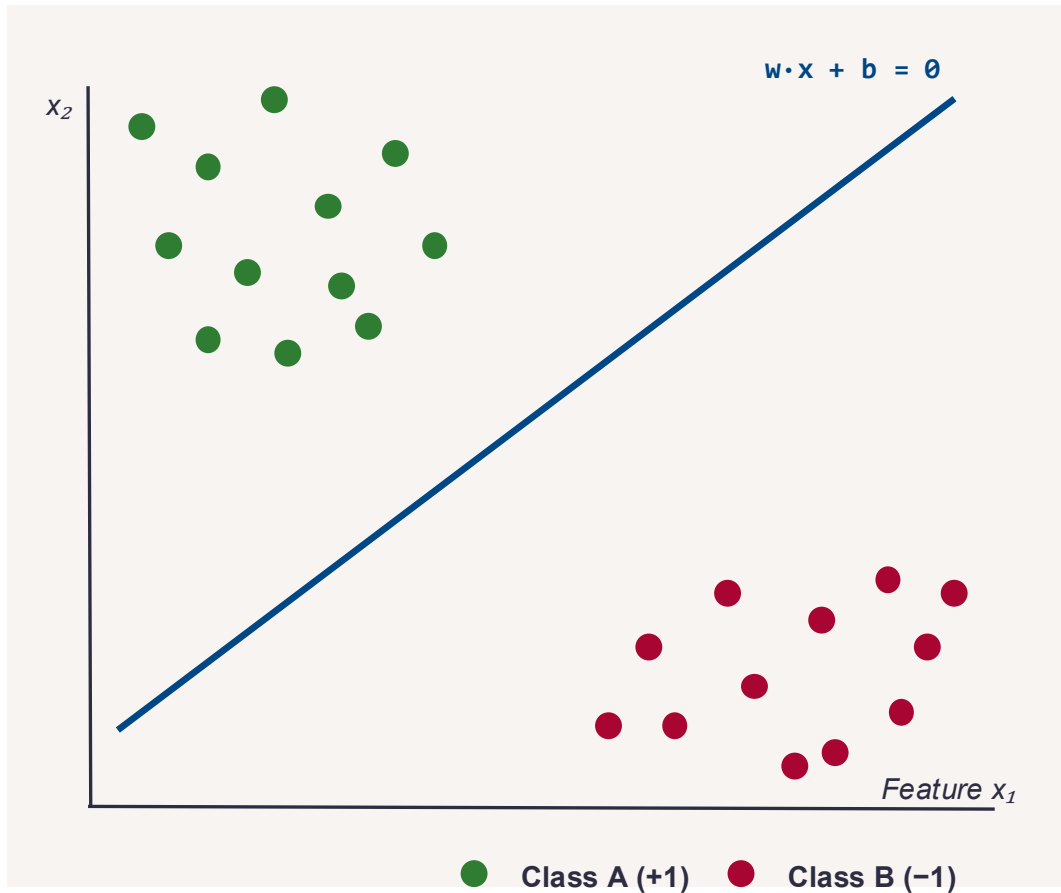
Use a separate validation set, simplify the model, add more data, regularize, or run k-fold cross-validation.

K-Fold Cross Validation



Linear Classifiers

Separate classes with a straight-line decision boundary



1

Decision rule

$\hat{y} = \text{sign}(w \cdot x + b) \rightarrow$ classify by which side of the line

2

Boundary is a hyperplane

A line in 2D, plane in 3D, hyperplane in higher dimensions

3

Training: find w and b

Pick parameters that best separate the two classes

4

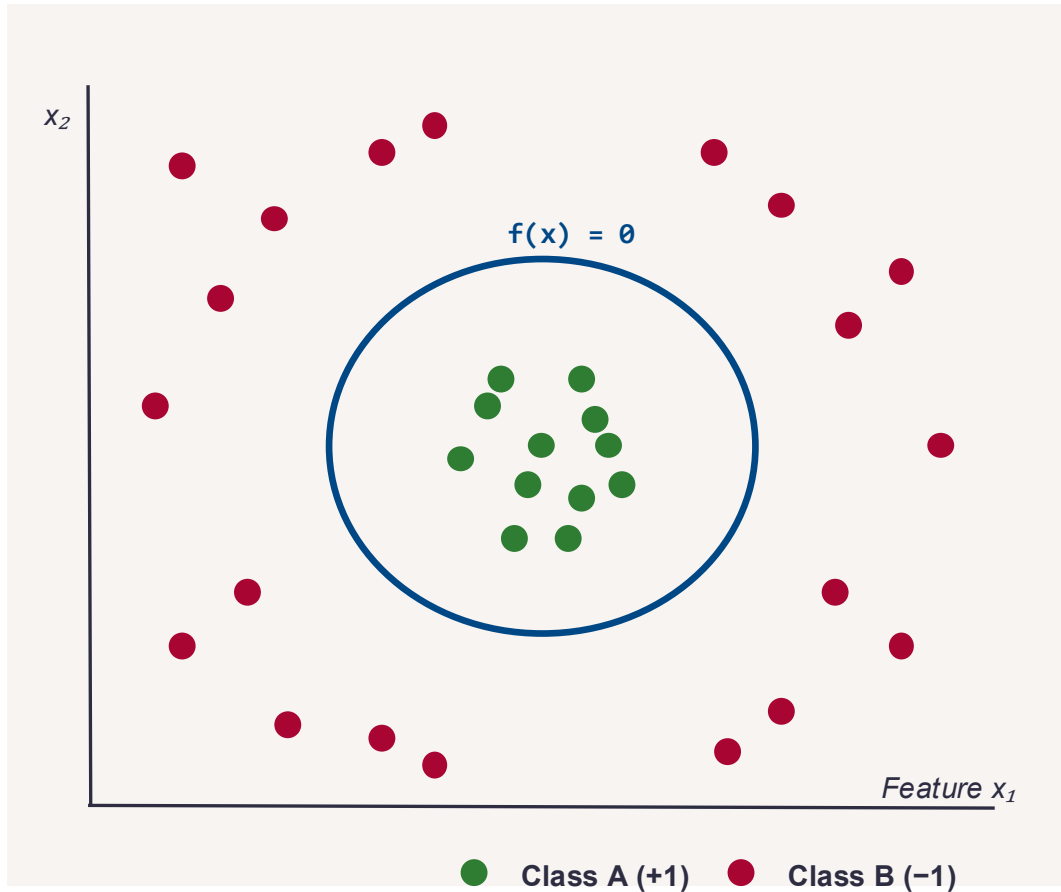
Common linear classifiers

Logistic Regression • Linear SVM

When classes aren't linearly separable, we need non-linear models — like decision trees

Non-Linear Classifiers

When a straight line isn't enough



1

Decision rule

$\hat{y} = \text{sign}(f(x)) \rightarrow f$ can be any non-linear function

2

Boundary is curved

Circles, polynomials, axis-aligned rectangles — whatever fits the data

3

Learns the shape from data

No straight-line assumption — model adapts to complex patterns

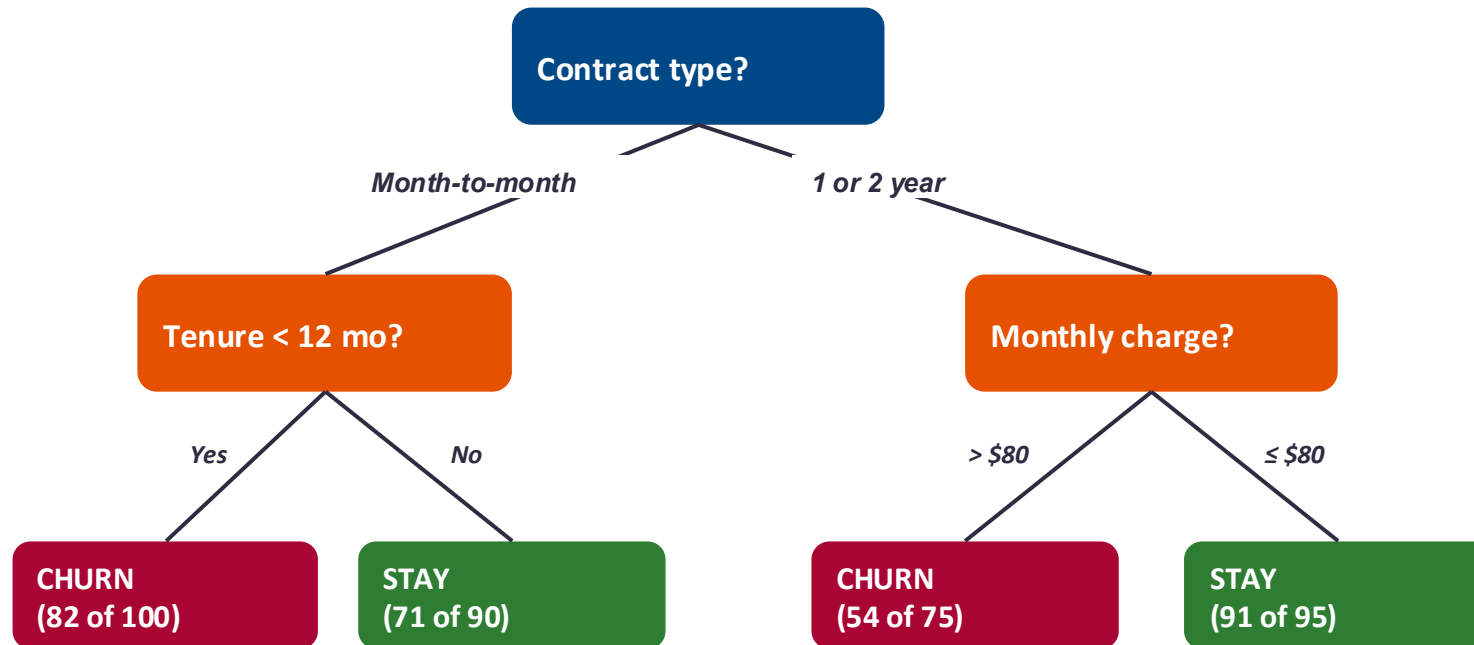
4

Common non-linear models

Decision Trees • Random Forests • Kernel SVM • Neural Networks

Decision Tree Example: Predicting Customer Churn

Will this customer cancel their subscription?



Customer Profile

Contract type:	Month-to-month
Tenure:	3 months
Monthly charge:	\$95
Tech support:	No

Prediction Path

- 1 Contract = Month-to-month → left
- 2 Tenure = 3 mo → < 12 → left

PREDICTION

CHURN (82% confidence)

Each leaf shows the majority class and how many training customers landed there.

Bootstrap Aggregation (Bagging)

How it works

1

Sample

Draw a random subset of training data (with replacement) and train one tree.

2

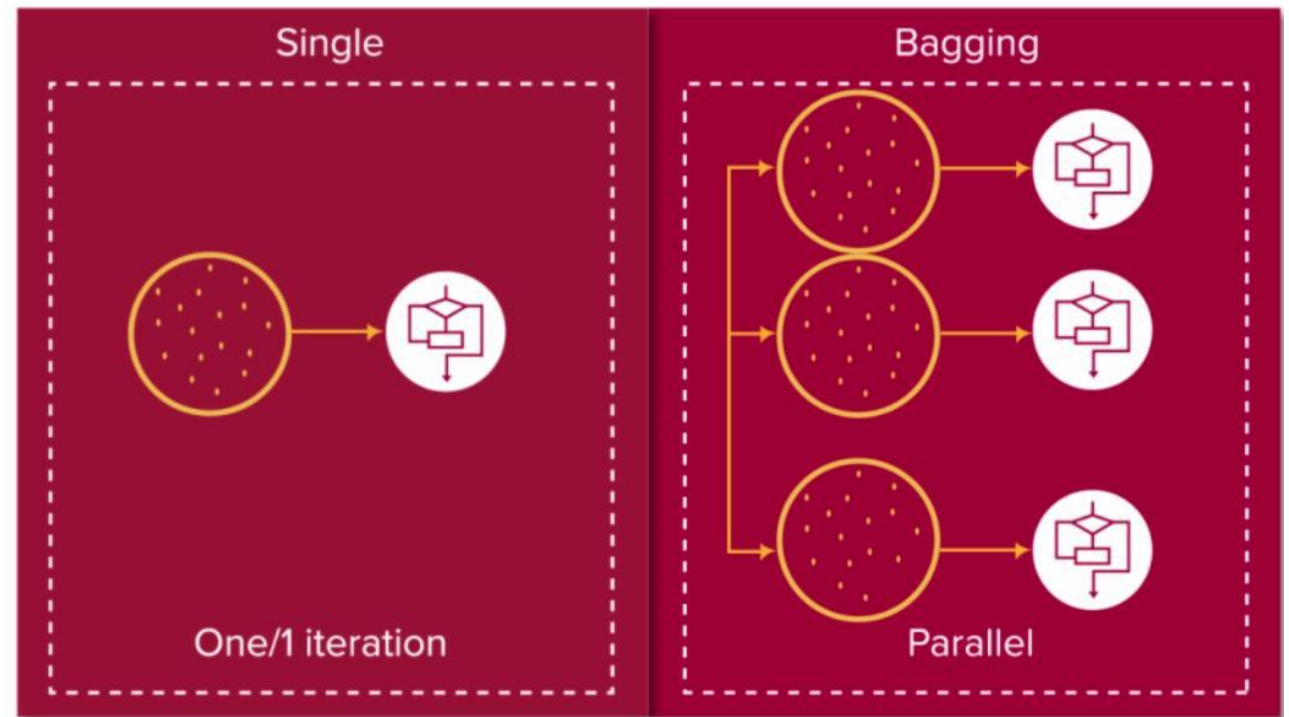
Repeat

Repeat with many independent samples — each produces its own tree.

3

Aggregate

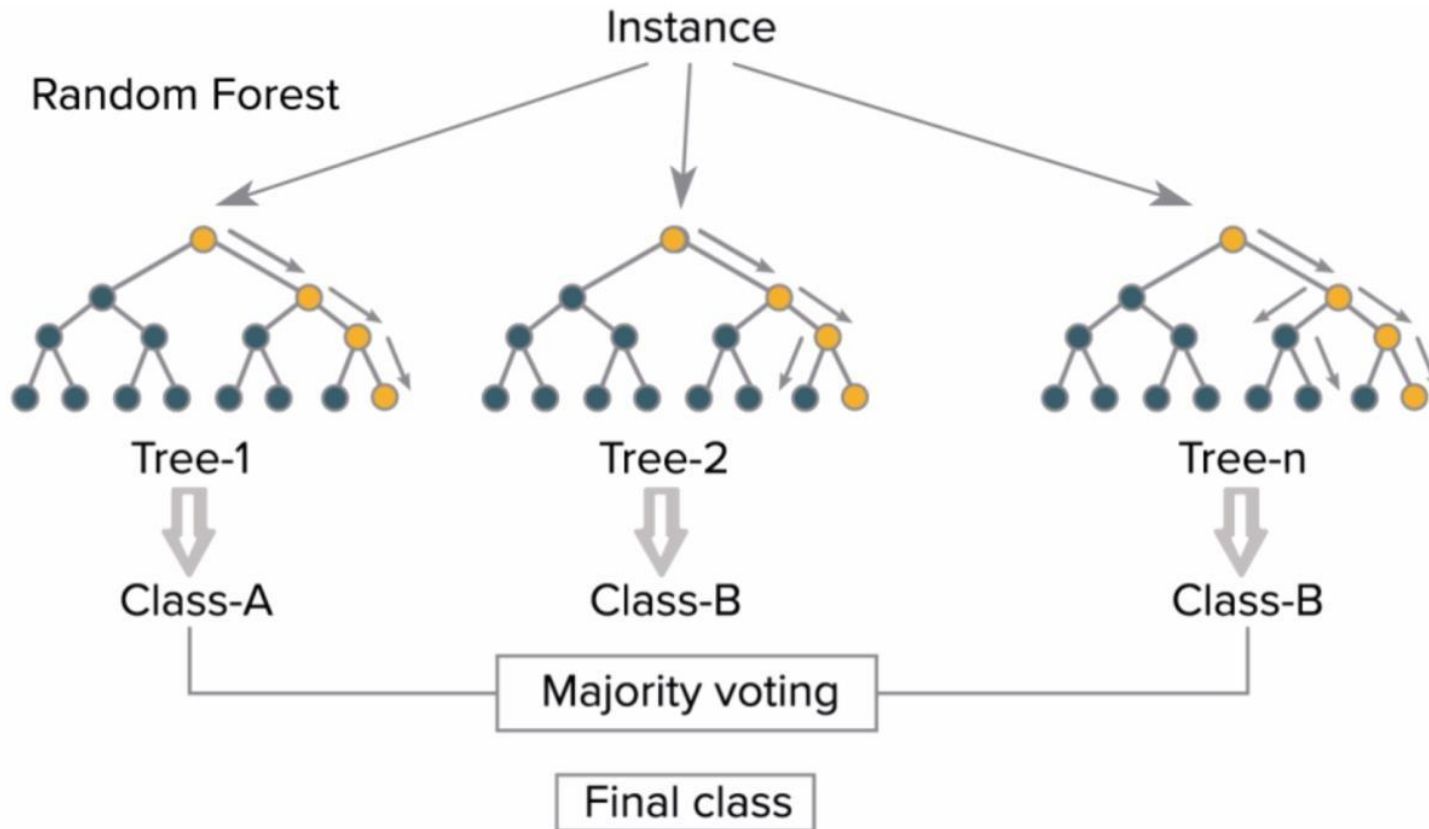
Combine predictions: majority vote for classification, average for regression.



Why it works

Single trees overfit to their training data. Averaging many trees cancels out errors → lower variance, more reliable predictions.

Random Forest



Why it works

- **Bagging + feature randomness.** Each split considers only a random subset of features.
- **Decorrelates trees.** Forces them to disagree, so errors cancel on the vote.
- **Lower variance.** Average of many noisy predictions beats any single tree.

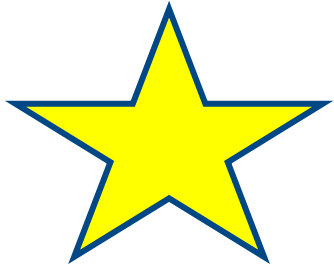
Key parameters

- **n_estimators** — number of trees (100–500 typical)
- **max_features** — features per split ($\approx \sqrt{p}$ for classification)
- **max_depth** — limits tree size to combat overfit

Tradeoffs

- ✓ **Robust** to noise and outliers; little tuning needed
- ✓ **Feature importance** comes for free
- ✗ **Slower** to train and predict than a single tree
- ✗ **Less interpretable** — no single tree to read

K-Nearest Neighbors



The star represents a new data point that we want to predict the class of (**red** or **blue**)

